

# Python/Django Developer Test

## Objective

Build a RESTful API with the Django REST Framework that manages ride information.

## Core Requirements

1. Use the Django REST Framework
  - Create models for Ride, User, and RideEvent.
  - Add serializers for each model so data can be converted to and from JSON.
  - Use ViewSets to handle CRUD operations.
2. Authentication
  - Restrict API access so that only users with the 'admin' role can call the endpoints.
3. Ride List API
  - Provide an endpoint that returns a list of Rides.
  - Each Ride in the response must include its related RideEvents and the related rider and driver (id\_rider, id\_driver).
  - Support pagination.
  - Support filtering by Ride status and by rider email.
  - Support sorting by pickup\_time and by distance to a given pickup GPS location, both within the same endpoint. Both sort options must be implemented as efficiently as possible, assuming the Ride table is very large. Pagination must still work when sorting is applied.
4. Performance
  - The RideEvent table is expected to grow very large, so the Ride List API must return an extra field on each Ride called todays\_ride\_events that contains only the RideEvents from the last 24 hours. For performance, the SQL Django generates must never load the full list of RideEvents. Use advanced Django features so that this data is fetched with as few SQL queries as possible.
  - Keep the total number of database queries for the Ride List API as low as possible. Returning the ride list with the related driver, rider, and RideEvents should be achievable in 2 queries (or 3 if you count the COUNT query used for pagination).
5. Table Definitions

## Ride Table

| Field            | Data Type | Description                                         |
|------------------|-----------|-----------------------------------------------------|
| id_ride          | INT       | Primary key                                         |
| status           | VARCHAR   | Ride status (e.g., 'en-route', 'pickup', 'dropoff') |
| id_rider         | INT       | Foreign key referencing User(id_user)               |
| id_driver        | INT       | Foreign key referencing User(id_user)               |
| pickup_latitude  | FLOAT     | Latitude of the pickup location                     |
| pickup_longitude | FLOAT     | Longitude of the pickup location                    |

| Field             | Data Type | Description                       |
|-------------------|-----------|-----------------------------------|
| dropoff_latitude  | FLOAT     | Latitude of the dropoff location  |
| dropoff_longitude | FLOAT     | Longitude of the dropoff location |
| pickup_time       | DATETIME  | Scheduled pickup time             |

### User Table

| Field        | Data Type | Description                        |
|--------------|-----------|------------------------------------|
| id_user      | INT       | Primary key                        |
| role         | VARCHAR   | User role ('admin' or other roles) |
| first_name   | VARCHAR   | User's first name                  |
| last_name    | VARCHAR   | User's last name                   |
| email        | VARCHAR   | User's email address               |
| phone_number | VARCHAR   | User's phone number                |

### Ride\_Event Table

| Field         | Data Type | Description                           |
|---------------|-----------|---------------------------------------|
| id_ride_event | INT       | Primary key                           |
| id_ride       | INT       | Foreign key referencing Ride(id_ride) |
| description   | VARCHAR   | Description of the ride event         |
| created_at    | DATETIME  | Timestamp of when the event occurred  |

## Submission

- Host the Django project source code in a version control repository (for example, GitHub).
- Keep the commit history clean and use meaningful commit messages that show how the project progressed.
- Provide a thorough README that lets any developer set up the project without trouble. Include any notes or comments about design decisions or challenges you ran into during implementation.

## Evaluation Criteria

- Functionality: Does the application meet every requirement above?
- Code Quality: Is the code modular, readable, and maintainable?
- Error Handling: Are errors and edge cases handled properly?
- Performance: Is the API tuned to make as few SQL queries as possible?

## Bonus - SQL

- For reporting, write a raw SQL statement (include it in the README) that returns the count of Trips whose duration from Pickup to Dropoff was more than 1 hour, grouped by Month and Driver.
- A sample of the expected report output is shown below.

### Sample Report

| Month   | Driver   | Count of Trips > 1 hr |
|---------|----------|-----------------------|
| 2024-01 | Chris H  | 4                     |
| 2024-01 | Howard Y | 5                     |
| 2024-01 | Randy W  | 2                     |
| 2024-02 | Chris H  | 7                     |
| 2024-02 | Howard Y | 5                     |
| 2024-03 | Chris H  | 2                     |
| 2024-03 | Howard Y | 2                     |
| 2024-03 | Randy W  | 11                    |
| 2024-04 | Howard Y | 7                     |
| 2024-04 | Randy W  | 3                     |

- When a driver picks up a rider, a Ride\_Event is created with the description 'Status changed to pickup'. When a rider is dropped off, a Ride\_Event is created with the description 'Status changed to dropoff'. To calculate the duration of a trip, find the pickup and dropoff RideEvents for each ride and compute the difference between their timestamps. (Note: You do not need to write the code that populates the Ride\_Event table — assume it is already populated when writing your SQL.)
- In a real production system, the trip distance would likely be stored directly on the Ride table, but for this assessment please assume the Ride table structure cannot be changed.